

Agentic Workflow Orchestrator

Multi-step LLM workflows that plan, run guarded steps, pause for human approval, and record a replayable audit trace.

A hands-on guide to get this project running from a clean machine, use every feature, and test it. Written so you (or future-you) can copy-paste straight through. Everything below was run against this repo and confirmed to work.

What this is

The **Agentic Workflow Orchestrator** is a small FastAPI engine that runs multi-step LLM workflows *safely*: it produces an explicit **plan** from a goal, runs it **one guarded step at a time** (schema validation + a step budget + retry-or-halt), **pauses for human approval** on consequential steps, and records everything as a **replayable audit trace**. The shipped demo is a neutral **research-to-brief** agent (retrieve → summarize → approve outline → cited brief).

Who it's for: anyone building multi-step automation with an LLM who needs it to be controllable and auditable rather than a black box.

It runs **cold with no API key** — the planner falls back to a deterministic built-in plan and the tools use a bundled **synthetic corpus**. Add an Anthropic key (or point at a local Ollama model) to plan with a real LLM.

Prerequisites

- **Python 3.11+** (this repo was verified on 3.13). Check with `python --version`.
- **pip** and the ability to create a virtualenv (`python -m venv`).
- ~150 MB disk for the virtualenv and dependencies.
- **Optional — make**. The `Makefile` gives short commands (`make run` , etc.). GNU Make is preinstalled on most macOS/Linux machines; on Windows it's often missing. Every `make` command below has a copy-pasteable **manual equivalent**, so you don't need `make`.
- **Optional — an LLM provider**. None required for the demo.
 - `ANTHROPIC_API_KEY` to plan with Claude (`MODEL_PROVIDER=anthropic` , default).
 - A running **Ollama** instance to plan locally (`MODEL_PROVIDER=ollama`).
 - With neither, planning uses the deterministic built-in planner.

No database to install — storage is a local **SQLite** file the app creates itself.

Setup

From a fresh clone (or copy) of the project, in the repo root (`agentic-workflow-orchestrator/`).

Option A — with `make`

```
make install      # create .venv and install dependencies
cp .env.example .env  # optional; only needed if you'll use a real LLM
make seed         # register demo tools + load the 2 sample goals
```

Option B — without `make`

macOS / Linux:

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
cp .env.example .env      # optional
.venv/bin/python -m app.seed
```

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env  # optional
.venv\Scripts\python.exe -m app.seed
```

About `.env`: it's optional. If you skip it, the app runs in deterministic mode. If you create it, the relevant keys are:

```
MODEL_PROVIDER=anthropic      # or "ollama"
ANTHROPIC_API_KEY=            # leave blank to stay in deterministic mode
ANTHROPIC_MODEL=claude-sonnet-4-6
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.1
DB_PATH=data/orchestrator.db  # SQLite file (auto-created, git-ignored)
```

A successful `make seed` prints the three registered tools (with `write_brief` marked `[consequential]`) and the two sample goals.

First run (guided happy path)

Start the server.

- **with make:** `make run`
- **without make (macOS/Linux):** `.venv/bin/python -m uvicorn app.main:app --reload`
- **without make (Windows):** `.venv\Scripts\python.exe -m uvicorn app.main:app --reload`

You should see Uvicorn report `Application startup complete` and `Uvicorn running on http://127.0.0.1:8000`. Open `http://localhost:8000`.

Now walk the research-to-brief flow (mirrors [DEMO.md](#)):

1. **You land on the goal page.** The text box is pre-filled with a seeded goal ("Write a short research brief on the benefits and risks of urban green roofs."). There's a **Browse the audit log** → link at the top and the list of allowlisted tools at the bottom.
2. **Click "Plan it."** A **Plan** card appears with **3 ordered steps** (`retrieve_documents` → `summarize_sources` → `write_brief`), each showing its inputs and expected output. The header reads `model: deterministic / builtin-deterministic-v1 · prompt: planner/v1` (or your Claude/Ollama model if configured). **Nothing has executed yet** — this is the "plan before it runs" moment.
3. **Click "Plan & run."** A **Run trace** card appears. Steps 1 and 2 show **succeeded** (with per-step timings), and the run **pauses** with a yellow ⚠ **Human checkpoint — step 3 is consequential** box and **Approve & continue / Reject** buttons. The executor stopped *before* the consequential `write_brief` step.
4. **Click "Approve & continue."** Step 3 runs, status flips to **completed**, and a **Final brief** section shows a cited markdown brief ending in a `## Sources` list. (Try **Reject** on another run: the run halts and the rejection is recorded.)
5. **Open the audit log.** Click the run id link, or go to `/runs` and open the run. You'll see the plan, every step with expandable **inputs/outputs** and timings, the recorded human decision, the full event trace with `+ms` offsets, and the brief. The `trace.json` link is the machine-readable replay artifact.

That's a complete, auditable run end to end.

CLI alternative (no browser)

Runs the whole flow headless, auto-approving the checkpoint, and prints the trace plus the brief:

```
# with make's venv already created:
.venv/bin/python -m app.cli "Write a short research brief on urban green roofs."      # macOS/
Linux
.venv\Scripts\python.exe -m app.cli "Write a short research brief on urban green roofs."  #
Windows
```

You should see `status: completed`, three `succeeded` steps, the event list, and a `--- BRIEF ---` section.

Feature by feature

Use the **seeded topics** for meaningful results — the synthetic corpus (`data/corpus/documents.json`) only covers **urban green roofs** and **community solar programs**.

1. Plan-only (inspect before running)

- **UI:** type a goal → **Plan it.** **CLI:** `python -m app.planner "<goal>"` (prints the plan as JSON).
- **Expect:** a 3-step plan referencing only the allowlisted tools, stamped with the model provider and prompt version. The plan is persisted even if you never run it.

2. Guarded stepwise execution

- **UI:** **Plan & run.** **CLI:** `python -m app.cli "<goal>"`.

- **Try:** "Summarize the current state of community solar programs for a policy brief."
- **Expect:** steps run one at a time, each output schema-validated, with timings; the run pauses at the consequential step (UI) or auto-approves (CLI).

3. Human checkpoint (approve / reject / resume)

- **UI:** on a paused run, click **Approve & continue** (completes) or **Reject** (halts). The decision is written to the audit trace as a `human_decision` event.
- **Expect:** approval produces the cited brief; rejection halts the run with step 3 marked `rejected`. A paused run survives a server restart (state is in SQLite).

4. Guardrail trip (a bad step is caught, not passed)

- **Try (UI or CLI):** "Write a brief on quantum teleportation hardware."
- **Expect:** `retrieve_documents` finds **no** matching documents (off-corpus), and the empty result is **rejected by the next step's input schema** — the run halts with status **failed** and a `validation_failure` event on step 2. The engine surfaces the guardrail instead of summarizing nothing.

5. Audit log & replay

- `/runs` — every run, newest first, with status.
- `/runs/{id}` — full viewer: plan, per-step inputs/outputs, checkpoint decision, event trace with timings, and the brief.
- `/runs/{id}/trace.json` — the complete trace as JSON (the replay artifact).

6. Health check

- `GET /health` → `{"status": "ok"}`. Handy for confirming the server is up: `curl http://localhost:8000/health`.

Testing it hands-on

The repo ships a **pytest** suite (37 tests) covering the registry/allowlist, the planner, the executor guardrails, checkpoints, the audit trace, and the route-level error states. There is **no separate eval/self-check target** in this repo — `make test` is the check.

```
make test
# or without make:
.venv/bin/python -m pytest -q          # macOS/Linux
.venv\Scripts\python.exe -m pytest -q  # Windows
```

How to read it: pytest prints one dot per passing test and a final summary line. "Good" looks like:

```
..... [100%]
37 passed in ~1s
```

Any **F** (failure) or **E** (error) is a problem — pytest prints the offending test and a traceback below the dots. The suite is hermetic (uses temporary SQLite databases and the deterministic planner), so it needs **no API key** and makes **no network calls**; it should pass on a clean machine right after `make install`.

For a quick manual end-to-end smoke test, run the CLI happy path and the guardrail trip and eyeball the output:

```
.venv/bin/python -m app.cli "Summarize community solar programs"      # completes  
.venv/bin/python -m app.cli "Write a brief on quantum teleportation hardware"  # halts: failed
```

Resetting

Return to a clean, seeded state (clears all runs/plans/steps/events, then re-registers tools and validates goals):

```
make reset  
# or without make:  
.venv/bin/python -m app.seed --reset      # macOS/Linux  
.venv\Scripts\python.exe -m app.seed --reset  # Windows
```

You'll see **Run-state cleared.** followed by the seed summary. The **/runs** list will be empty afterward. (Plain **make seed** / **python -m app.seed** initializes storage and re-validates **without** clearing existing runs.)

Troubleshooting

Symptom	Cause & fix
<code>make: command not found</code> (often Windows)	<code>make</code> isn't installed. Use the without-make manual commands in each section above.
Plan shows <code>model: deterministic / builtin-deterministic-v1</code> when you expected Claude	No usable <code>ANTHROPIC_API_KEY</code> . This is normal/intended for a cold demo. To use Claude, set <code>MODEL_PROVIDER=anthropic</code> and a valid key in <code>.env</code> , then restart the server.
Model name shows <code>(fallback: ...)</code>	The configured provider failed (bad key, Ollama not running, network) and the planner fell back to deterministic so the demo keeps working. Fix the provider/ key and restart, or stay deterministic.
<code>[Errno 48/98] address already in use / port 8000 busy</code>	Another process holds port 8000. Run on another port: <code>... -m uvicorn app.main:app --reload --port 8001</code> and open <code>http://localhost:8001</code> .
Run ends <code>failed</code> with a <code>validation_failure</code> on step 2 / empty brief	Your goal is off-corpus , so retrieval found nothing. Use a goal about the seeded topics (urban green roofs or community solar). This is the guardrail working as designed.
No seeded goal in the text box / empty tools list	Run <code>make seed</code> (or <code>python -m app.seed</code>) and make sure <code>data/demo_goals.seed.json</code> and <code>data/corpus/documents.json</code> exist. Restart the server.
<code>ModuleNotFoundError: app</code> or wrong Python	Run commands from the repo root , and use the venv's interpreter (<code>.venv/bin/python</code> / <code>.venv\Scripts\python.exe</code>) or activate the venv first (<code>source .venv/bin/activate</code> / <code>.venv\Scripts\Activate.ps1</code>).
Old runs cluttering <code>/runs</code> , or odd DB state	<code>make reset</code> (clears all run-state and re-seeds).
<code>Couldn't proceed</code> card after submitting	Expected for an empty goal or an invalid action (e.g., resuming a run with no pending checkpoint). Adjust the goal and try again.

Data & safety note

- **Synthetic/public data only.** Retrieval runs over a small bundled synthetic corpus (`data/corpus/documents.json`); there is **no PHI and no connection to any internal system**. The model + prompt version is recorded on every plan.
- **The planner proposes; guardrails and humans dispose.** Tools are allowlisted, every step's input/output is schema-validated, steps are budgeted, and the consequential `write_brief` step requires explicit human approval.
- **Everything is auditable** — each run is a replayable trace (`/runs/{id}` and `trace.json`).
- **Using real or sensitive data would require additional controls** not in this prototype: a richer per-tool permission/cost policy, tool sandboxing, role-based approvals, full audit retention, and an appropriate data-handling boundary (e.g., a BAA / in-boundary model) for any sensitive-data tool. See **Path to production** in `README.md`.

Reference docs: [README.md](#) (architecture), [DEMO.md](#) (demo script), [DECISIONS.md](#) (why each choice was made), [TODO.md](#) (phased build status — all phases shipped).